RED4SEC

Security Audit Report

06/20/2026

xGAS

RED4SEC

Content

Introduction

xGAS is a solidity implementation of a wrapped native asset. The system incorporates a standard ERC-20-compatible wrapping layer that enables deposit and withdrawal operations, while also adding signature-based authorization mechanisms to support more flexible and secure interactions. In particular, it integrates **ERC-2612** permit flows and **EIP-3009** transfer authorizations, preserving the canonical *(v, r, s)* interfaces while also providing alternative **bytes-based signature** paths compatible with both externally owned accounts (**EOAs**) and smart contract wallets implementing **ERC-1271**.

As requested by **AxLabs**, and as part of the vulnerability review and management process, Red4Sec was engaged to perform a security code audit to evaluate the security of the **xGAS** project.

This report includes the findings identified during the audit of **xGAS**. The analysis performed shows that the smart contract does not contain any critical vulnerabilities.

Disclaimer

This document only represents the results of the code audit conducted by Red4Sec Cybersecurity and should not be used in any way to make investment decisions or as investment advice on a project.

Likewise, the report should not be considered either "endorsement" nor "disapproval" of the guarantee of the correct business model of the analyzed project, nor as guarantee on the operation or viability of the implemented financial product.

Red4Sec makes full effort and applies every resource available for each audit, however it does not warrant the function, nor the safety of the project and it cannot be deemed a sufficient assessment of the code's utility and safety, bug-free status, or any other declarations of the project. Additionally, Red4Sec makes no security assessments or judgments about the underlying business strategy, or the individuals involved in the project.

Blockchain technology and cryptographic assets come with their own new risks and challenges, where the ecosystem, platform, its programming language, and other software related to said technology can have vulnerabilities that could lead to exploits. As a result, the audit cannot guarantee the explicit security of the audited projects.

The audit reports can be used to improve the code quality of smart contracts, to help limit the vectors of attack and to lower the high level of risks associated with utilizing new and continually changing technologies such as cryptographic tokens and blockchain, but they are unable to detect any future security concerns with the related technologies.

Audit Scope

Red4Sec Cybersecurity has conducted a thorough audit of the security level of **xGAS** against potential attacks, identifying possible errors in its design, configuration, or implementation. The objective of the assessment was to evaluate the availability, integrity, and confidentiality of the project and of any assets processed or stored by it.

The scope of this evaluation includes the following items provided by AxLabs:

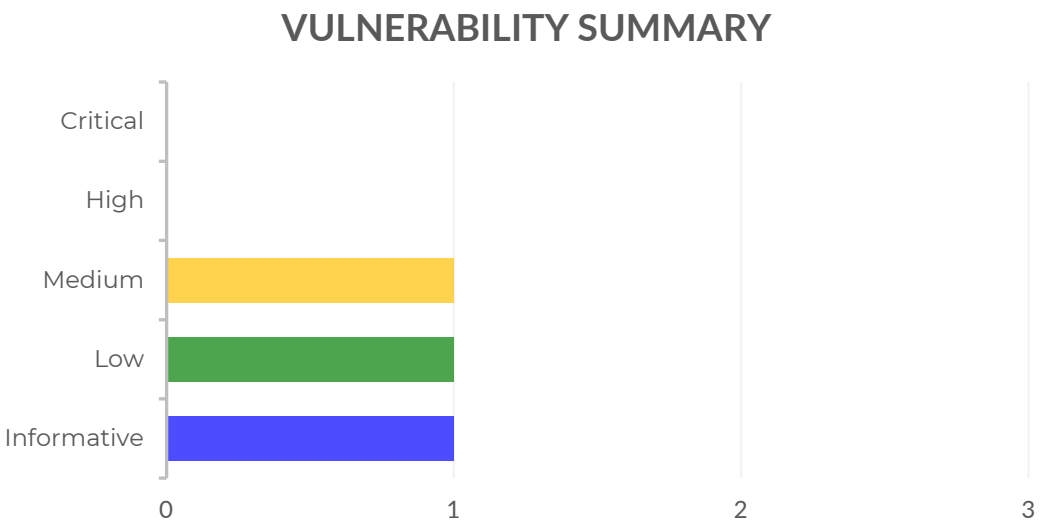
- https://github.com/bane-labs/xgas
    - Commit: `660c47977ab34abab2172c35d4cfc25ee4a77194`

Executive Summary

The security audit against **xGAS** was conducted between the following dates: **04/20/2026** and **04/22/2026**. A review of the remediation fixes was subsequently performed on **05/20/2026**.

Once the analysis of the technical aspects was completed, the results showed that the audited source code contained non-critical vulnerabilities.

During the analysis, a total of 3 vulnerabilities were identified. These issues have been classified according to the risk levels defined in the Vulnerabilities Severity Annex and have already been addressed.



Conclusions

After conducting a thorough audit of the smart contract, the codebase shows a solid security baseline and a clear architecture. No critical or high-severity vulnerabilities were identified that would enable direct unauthorized fund extraction under the current design assumptions. Overall, **the protocol is considered production-ready from a security perspective**, and the issues identified during the audit have been remediated.

The main security-relevant finding is **EIP-7702-sensitive signature semantics**. This was not a coding bug in isolation, but a real integration risk introduced by the interaction between dynamic signer type detection (`EOA` vs `ERC-1271`) and Prague/EIP-7702 account delegation. In practice, this could create behavioral changes for `bytes-based` signature flows if an account transitioned to delegated code. The issue has been addressed through explicit documentation, signer-mode-aware testing, and clear integration rules for relayers and frontends.

A secondary finding was **partial EIP-3009 compatibility for smart contract wallets**. In this project, smart-wallet support was present for transfer/receive authorization through the encoded-signature extension, while cancellation remained on the classic `(v,r,s)` path. This was primarily a feature-coverage and interoperability limitation rather than a direct exploit vector. Cancellation support has been extended to an encoded-signature path, improving consistency and UX.

Additional observations were primarily operational and design trade-offs rather than direct vulnerabilities, including the intentionally immutable/no-emergency-control model, potential native-asset surplus accounting drift from forced transfers, standard ERC-20 allowance race semantics, and dependency hygiene considerations around branch-based submodule references outside the locked Foundry workflow.

In summary, the protocol is secure by design for its intended scope, with no severe security blockers identified. The remediation of the identified medium/low-priority issues has further strengthened robustness, integrator safety, and future-proof behavior as account models evolve.

Vulnerabilities

In this section, you can find a detailed analysis of the vulnerabilities encountered upon the security audit.

List of vulnerabilities

Below, we have gathered a complete list of the vulnerabilities detected by Red4Sec, presented, and summarized in a way that can be used for risk management and mitigation.

Table of vulnerabilities			
ID	**Vulnerability**	**Risk**	**State**
XGAS-01	EIP-7702-Sensitive Signature Semantics	**Medium**	**Assumed**
XGAS-02	Partial EIP-3009 Compatibility	**Low**	**Fixed**
XGAS-03	Additional Security Considerations	**Informative**	**Fixed**

Vulnerability details

In this section, we provide the details of each of the detected vulnerabilities indicating the following aspects:

- Category
- Active
- Risk
- Description
- Recommendations

EIP-7702-Sensitive Signature Semantics

Identifier	Category	Risk	State
XGAS-01	Design Weaknesses	**Medium**	**Assumed**

xGAS validates encoded signatures using `SignatureChecker.isValidSignatureNow(...)`, whose validation semantics change dynamically depending on whether the signer address contains code.

The function selects the validation method based on account code presence:

- if `signer.code.length == 0`, it validates as an **EOA** using ECDSA,
- if `signer.code.length > 0`, it validates as a **smart contract** under ERC-1271.

With EIP-7702 (Prague), currently implemented in the **NeoX** EVM, a user address can delegate code to its own account. Consequently, **a single address may transition from "EOA mode" to "contract mode" between the time of signing and execution**.

In **xGAS**, this behavior affects extended pathways:

- `permit(..., bytes signature)` in `ERC20Permit1271`,
- `transferWithAuthorization(..., bytes signature)` and `receiveWithAuthorization(..., bytes signature)` in `TransferAuth`.

As a result, a signature that is valid under EOA semantics may become invalid if the account delegates code afterward, and vice versa.

This is not an isolated defect in **xGAS** logic, but rather an emergent effect of:

1. the `SignatureChecker` design, which dynamically switches between EOA and ERC-1271 validation based on `code.length` and
2. the capabilities introduced by EIP-7702, allowing accounts to alter their cryptographic behavior at a single address.

The primary operational impacts and risks are as follows:

- Functional denial of service affecting pending signatures, particularly in relayer-based flows or off-chain signed authorizations. Bytes signatures issued prior to code delegation may fail upon execution if the delegated ERC-1271 logic does not accept them.

- Expanded abuse surface through insecure delegation. If a user delegates to a contract with weak ERC-1271 validation, such as permissive signature acceptance, the effective security of the account degrades to that of the delegated contract.

- Non-uniform behavior across pathways. In this project, classic `(v, r, s)` EIP-3009 pathways rely on direct `ECDSA.tryRecover`, while encoded signature pathways use `SignatureChecker`. Consequently, the same address may yield different validation outcomes depending on the input format.

Recommendations

- Explicitly document that, under EIP-7702, the security of `bytes signature` pathways depends on the code delegated by the signing account.
- Add tests covering the transition of an address between EOA mode and ERC-1271 mode.
- Consider adding UX and SDK warnings, as well as additional validation in relayers, SDKs, or interfaces.

Source Code References

- foundry.toml#L8
- src/extensions/ERC20Permit1271.sol
- src/extensions/TransferAuth.sol#L12

Fixes Review

This issue has been addressed in the following pull request, where the development team acknowledges the finding as a design-level compatibility risk and has properly documented the use of the contract:

- https://github.com/bane-labs/xgas/pull/13

Developer Team Response

"The bytes-signature entrypoints intentionally use `SignatureChecker.isValidSignatureNow` to support both EOAs and ERC-1271-compatible accounts. Under EIP-7702, an account may change from EOA-like validation semantics to delegated-code validation semantics at the same address. This can affect pending off-chain authorizations that were signed before delegation and executed afterward.

We do not plan to add an unconditional ECDSA-first fallback before ERC-1271 validation. While that could preserve some pre-delegation EOA signatures, it would also allow raw ECDSA signatures to bypass the delegated account's current ERC-1271 validation policy. For bytes-signature entrypoints, xGAS intentionally follows the signer account's current validation semantics.

Users and relayers that require strict ECDSA-only validation should use the classic EIP-3009 `(v, r, s)` entrypoints. The bytes-signature entrypoints should be treated as account-abstraction-aware paths whose behavior depends on the signer's current account code.

This behavior has been documented, including its effect on pending bytes-signature authorizations under EIP-7702. Signer-mode-aware tests were added for EOA-mode and ERC-1271-mode validation, together with integration guidance for SDKs, relayers, and frontends."

Partial EIP-3009 Compatibility

Identifier	Category	Risk	State
XGAS-02	Design Weaknesses	**Low**	**Fixed**

The implementation provides partial compatibility with smart contract wallets in EIP-3009 flows, as authorized transfers support `bytes signature`, while revocation through `cancelAuthorization` remains limited to the classic `(v, r, s)` format.

The `EIP3009.sol` core validates signatures through ECDSA recovery, a model designed for traditional EOAs. Although `TransferAuth` extends `transferWithAuthorization` and `receiveWithAuthorization` to support smart wallets through `SignatureChecker` and ERC-1271, no equivalent pathway exists to cancel authorizations using encoded signatures. As a result, a smart contract wallet can authorize transfers through the extended pathway, but lacks the same capability to revoke them off-chain with an ERC-1271 signature.

This may create an inconsistent experience for integrators and smart wallet users, while also limiting revocation capabilities in relayer-based flows. The impact is primarily functional and compatibility-related, rather than a critical vulnerability causing direct loss of funds.

Recommendations

- Add a `cancelAuthorization` variant that accepts `bytes signature` and validates it through `SignatureChecker`.
- Maintain consistency between authorization and cancellation pathways for EOAs and smart contract wallets.
- Explicitly document that signed cancellation of EIP-3009 authorizations is only supported for classic ECDSA signatures.

References

- https://eips.ethereum.org/EIPS/eip-7598
- https://github.com/TheGreatAxios/eip3009-forwarder

Source Code References

- src/extensions/EIP3009.sol#L86

Fixes Review

This issue has been addressed in the following pull request, where the development team has added the missing method:

- https://github.com/bane-labs/xgas/pull/12

Additional Security Considerations

Identifier	Category	Risk	State
XGAS-03	Codebase Quality	**Informative**	**Fixed**

The following additional security considerations were identified during the review. These issues do not currently represent directly exploitable vulnerabilities, but they introduce operational, integration, or design risks that should be assessed to ensure the system's behavior remains aligned with the project's requirements.

These behaviors are known and, in some cases, reflect reasonable design decisions. Nevertheless, they should be explicitly documented to prevent incorrect assumptions by integrators, frontends, or users.

Lack of Emergency Controls

The core contracts do not implement pause mechanisms, kill switches, or incident response controls.

In the event of a critical failure, an exploit in ecosystem dependencies, or an incident arising from external integrations, no on-chain mechanism would be available to temporarily contain sensitive operations such as deposits, withdrawals, or authorizations. This design choice reduces the administrative abuse surface, but also limits the team's ability to respond to unforeseen events.

Forced GAS Desynchronization

The contract maintains an expected relationship between the xGAS supply and the custodied GAS balance, but this relationship may be altered by forced GAS transfers to the contract.

Under the normal flow, `deposit` mints exactly `msg.value` and `withdraw` burns exactly `amount` before transferring GAS, so no critically exploitable desynchronization was observed. In addition, the absence of `payable receive()` and `fallback()` functions prevents accidental transfers. However, as with any native-asset wrapper, the contract may receive GAS forcibly through mechanisms such as `selfdestruct`, coinbase rewards, or funds sent to a precomputed address before deployment.

This surplus would not allow users to withdraw more GAS than the amount corresponding to the tokens burned, but it could create accounting discrepancies between `totalSupply()` and `address(this).balance`. Since there is no administrative sweep function, such surplus funds would remain trapped in the contract unless the project design explicitly accounts for this scenario.

ERC-20 Approval Race

The implementation inherits the standard ERC-20 `approve(spender, value)` behavior, where an existing allowance is directly overwritten by the new value.

This behavior is well known in the ERC-20 standard and may result in a race condition when a user attempts to reduce an existing allowance. A malicious `spender` could consume the previous allowance before the update transaction is confirmed and subsequently use the new allowance as well, causing cumulative spending greater than the user intended.

This is not an xGAS-specific bug, but rather a classical limitation of the ERC-20 standard. Since recent OpenZeppelin versions do not expose `increaseAllowance` and `decreaseAllowance` by default, mitigation should rely on the usage patterns recommended by integrators, interfaces, and users.

Unpinned Git Submodules

The `.gitmodules` file declares external dependencies through references to generic branches, such as `main` or `master`, rather than pinning them to specific immutable commits.

Although `foundry.lock` enables reproducible dependency resolution within the standard Foundry workflow, `.gitmodules` remains Git's native mechanism for managing submodules. Therefore, if a developer, external fork, or CI/CD pipeline executes commands such as `git submodule update --init --recursive` outside the controlled Foundry workflow, more recent dependency versions may be downloaded without prior review.

This behavior introduces a supply chain risk, as the code being compiled or analyzed may vary depending on the time of retrieval. Although the risk appears limited, supply chain attacks have increased substantially in recent months.

Recommendations

- Assess whether the project's operational model requires a pause or kill switch mechanism.
- Document that contract balance may exceed `totalSupply()` due to forced GAS transfers.
- Document that allowance updates should follow the `approve(spender, 0)` pattern before setting a new value.
- Pin `.gitmodules` submodules to specific reviewed commits, avoiding dynamic branch references.

Fixes Review

These issues have been addressed in the following pull request:

- https://github.com/bane-labs/xgas/pull/14

Annexes

Methodology

A code audit is a thorough examination of the source code of a project with the objective of identifying errors, discovering security breaches, or contraventions of programming standards. It is an essential component to the defense in programming, which seeks to minimize errors prior to the deployment of the product.

Red4Sec adopts a set of cybersecurity tools and best security practices to audit the source code of the smart contract by conducting a search for vulnerabilities and flaws.

The audit team performs an analysis on the functionality of the code, a manual audit, and automated verifications, considering the following crucial features of the code:

- The implementation conforms to protocol standards and adheres to best coding practices.
- The code is secure against common and uncommon vectors of attack.
- The logic of the contract complies with the specifications and intentions of the client.
- The business logic and the interactions with similar industry protocols do not contain errors or lead to dangerous situations to the integrity of the system.

In order to standardize the evaluation, the audit is executed by industry experts, in accordance with the following procedures:

Manual Analysis

- Manual review of the code, line-by-line, to discover errors or unexpected conditions.
- Assess the overall structure, complexity, and quality of the project.
- Search for issues based on the SWC Registry and known attacks.
- Review known vulnerabilities in the third-party libraries used.
- Analysis of the business logic and algorithms of the protocol to identify potential risk exposures.
- Manual testing to verify the operation, optimization, and stability of the code.

Automatic Analysis

- Scan the source code with static and dynamic security tools to search for known vulnerabilities.
- Manual verification of all the issues found by the tools and analyzes their impact.
- Perform unit tests and verify the coverage.

Vulnerabilities Severity

Red4Sec determines the severity of vulnerabilities found in risk levels according to the impact level defined by CVSSv3 (Common Vulnerability Scoring System) by the National Institute of Standards and Technology (NIST), classifying the risk of vulnerabilities on the following scale:

Severity	Description
Critical	Vulnerabilities that possess the highest impact over the systems, services and/or sensitive information. The existence of these vulnerabilities is dangerous and should be fixed as soon as possible.
High	Vulnerabilities that could compromise severely compromise the service or the information it manages even if the vulnerability requires expertise to be exploited.
Medium	Vulnerabilities that on their own can have a limited impact and/or that combined with other vulnerabilities could have a greater impact.
Low	These vulnerabilities do not suppose a real risk for the systems. Also includes vulnerabilities which are extremely hard to exploit or whose impact on the service is low.
Informative	It covers various characteristics, information or behaviours that can be considered as inappropriate, without being considered as vulnerabilities by themselves.

Invest in Security, invest in your future